



PLAN DE TRABAJO

Safety Control Tower

[Alcance inicial presentado por Sponsor](#)
[Cotización aceptada N°4 - 21-07-2026](#)
[Carpeta Google Drive Proyecto](#)

Versión actualizada — 08 de septiembre de 2026
por [Juan Antonio Concha Loyola](#) - CTO www.helheim.cl

Reemplaza la versión:

07-08-2026 — actualizado para reflejar el estado real de las 3 etapas del proyecto.

08-08-2026 - incorpora el cierre técnico de Etapa 1 y el inicio efectivo del desarrollo funcional de Etapa 2.

Resumen ejecutivo

El objetivo del proyecto es diseñar, desarrollar e implementar una plataforma web responsiva que centralice la gestión de seguridad, cumplimiento y trazabilidad operacional, proporcionando una solución modular, escalable y multiplataforma que optimice los procesos, facilite la toma de decisiones y asegure la operación mediante un modelo de entrega incremental con validación temprana de usuarios.

Estrategia de implementación: desarrollo incremental con un MVP operativo para validar el modelo con usuarios reales, reducir riesgos de implementación e incorporar mejoras tempranas.

Plazo de ejecución: 3 meses (12 semanas), distribuidos en tres etapas secuenciales con entregables definidos y un esquema formal de seguimiento y control.

Módulo / Actividad	Estado	Responsable
Etapas 1 - Desarrollo MVP	RESUELTA	FP original: 28-ago
Etapas 2 - Desarrollo Core Funcional	EN CURSO	FP original: 30-sep
Etapas 3 - Optimización y Escalabilidad	PENDIENTE <i>inicia fin de septiembre</i>	FP original: 30-oct

Inicio formal del proyecto (01-08-2026)

Tecnología: PHP, MySQL y Css3, utilizando Scrum como metodología de gestión para desarrollar una solución robusta, modular y escalable. De ser necesarios Frameworks para agilidad, serán Laravel y Bootstrap.

Stack real en uso (actualizado — ver Registro de cambios de alcance N°3):

- Backend: PHP nativo con PDO (no Laravel). Sentencias preparadas en todas las consultas, nunca SQL concatenado con datos de usuario.
- Base de datos: MySQL/MariaDB — safetyco_SCT, servidor 201.148.104.98 (migrada desde tecaivot_SCT en 201.148.105.87 — ver Registro de cambios de alcance N°5). Cotejamiento utf8mb4_unicode_ci para compatibilidad entre los 5 idiomas soportados.
- Frontend: Bootstrap 5 + JavaScript vanilla (sin framework de frontend).
- Generación de PDF: Dompdf, conectado al flujo real de certificados de inducción.
- Metodología de gestión: Scrum, alineado a buenas prácticas de PMBOK (modelo híbrido predictivo/ágil).
- Versión de entregas: convención vs1, vs2, vs3... mantenida como estándar formal del proyecto

Alcance del servicio: comprende todo el ciclo de vida del producto digital, desde la planificación estratégica y el diseño UX/UI hasta el desarrollo, puesta en producción, estabilización y mejora continua.

Cambio de fondo respecto de la versión original

Durante la ejecución, el proyecto pasó de ser "un sitio de Tecaivot con SCT como uno de sus productos" a ser dos marcas y dos sitios completamente separados:

- www.safetycontrolltower.cl — enfocado 100% en el producto SCT, con su propia identidad visual (logo, favicon, paleta de colores).
- www.tecaivot.cl — sitio propio de la empresa, con su paleta e identidad original, con un enlace destacado hacia Safety Control Tower para quien busque información del producto.

El detalle completo de esta y otras decisiones de alcance está en la sección Registro de cambios de alcance.

Control de versiones en desarrollo

Etapas 1:

vs1: https://tecaivot.cl/dev/tecaivot_SCT_E1/vs1/index.html

vs2: https://tecaivot.cl/dev/tecaivot_SCT_E1/vs2/index.html

vs3: https://tecaivot.cl/dev/tecaivot_SCT_E1/vs3/index.php

vs4: https://tecaivot.cl/dev/tecaivot_SCT_E1/vs4/index.php

<https://tecaivot.cl/> -> quedo exclusivo como sitio

<https://www.safetycontrolltower.cl/> -> quedo exclusivo como sitio del proyecto

Etapas 2:

* vs1: https://www.safetycontrolltower.cl/dev/tecaivot_SCT_E2/vs1/index.php

vs2: https://www.safetycontrolltower.cl/dev/tecaivot_SCT_E2/vs2/index.php

* indica la version instalada de frente al usuario en <https://www.safetycontrolltower.cl/>

Perfiles de prueba para usuarios fuera del sistema.

- Usuario: GerenteEmpresaDemo@demoSCT.cl
- Usuario: adminEmpresaDemo@demoSCT.cl
- Usuario: JefaturaEmpresaDemo@demoSCT.cl
- Usuario: UsuarioEmpresaDemo@demoSCT.cl
- pass: DemoSCT_2026

Índice

Resumen ejecutivo

Cambio de fondo respecto de la versión original

Índice

Registro de cambios de alcance - SAFETY CONTROL TOWER (SCT)

1. Sitio público de Tecaivot multi-idioma

2. Login sin contraseña (código por correo)

3. Stack tecnológico: PHP nativo en vez de Laravel

4. Expansión de roles de 3 a 5 en gestión de usuarios

5. Migración de servidor y base de datos

6. Separación de marca: Tecaivot y SCT en sitios independientes

7. Tabla nueva no contemplada en el diccionario original: security_event_evidence

Etapas del servicio

Etapa 1 – Desarrollo MVP (FP 28 agosto)

• Credenciales de acceso al hosting www.tecaivot.cl

• Credenciales de acceso al hosting www.safetycontroltower.cl

• Manual de Marca

• Infraestructura y arquitectura base

• Módulos funcionales

Etapa 2 – Desarrollo Core Funcional (FP 30 de Septiembre)

Etapa 3 – Optimización y Escalabilidad (FP 30 de Octubre)

Anexo Diccionario de base de datos - SAFETY CONTROL TOWER (SCT)

ÍNDICE — 34 tablas documentadas

Registro de cambios de alcance - SAFETY CONTROL TOWER (SCT)

Registro de decisiones tomadas durante la ejecución que modifican o desvían del alcance originalmente descrito en este documento.

1. Sitio público de Tecaivot multi-idioma

Fecha: 21-08-2026. Solicitado por: Francisco Fredes (Sponsor).

El index.php público se reconstruyó para presentar a Tecaivot como empresa de servicios y desarrollo de herramientas TI (no específico de SCT), con SCT mostrado como uno de sus productos, y soporte completo para 5 idiomas (español, inglés, portugués, francés, chino)

2. Login sin contraseña (código por correo)

Fecha: 21-08-2026. Solicitado por: Francisco Fredes (Sponsor).

El mecanismo de autenticación se reemplazó de correo + contraseña a correo + código de un solo uso enviado por email, manteniendo el mismo nivel de rigor de seguridad (protección contra fuerza bruta, imposibilidad de saber qué correos están registrados, expiración de 10 minutos). Deja obsoleto el requisito "Recuperación de contraseña" del módulo de autenticación original.

Autenticación reforzada con contraseña + segundo factor por correo

Fecha: 08-09-2026. Solicitado por: Francisco Fredes (Sponsor).

"El mecanismo de autenticación fue actualizado a un esquema de doble factor: correo electrónico + contraseña como primer factor y código OTP enviado por correo como segundo factor. Se implementó provisión, activación y restablecimiento seguro de contraseña para cuentas nuevas y existentes. Los hashes de contraseña fueron desacoplados de la tabla users y almacenados en user_credentials. Las credenciales heredadas basadas en RUT fueron invalidadas y no se reutilizan."

"Cinco intentos fallidos de contraseña dentro de 10 minutos producen un bloqueo de 15 minutos. Los accesos correctos no contabilizan para este límite. Los intentos OTP mantienen su control independiente."

3. Stack tecnológico: PHP nativo en vez de Laravel

Fecha: 20-08-2026.

El desarrollo actual no utiliza el framework Laravel especificado en la Cotización y el Contrato, sino PHP nativo con PDO. Decisión validada entre el CTO y sponsor del proyecto

4. Expansión de roles de 3 a 5 en gestión de usuarios

Fecha: 24-08-2026.

El sistema de roles pasó de 3 (administrador, cliente, trabajador) a 5, agregando administrador_completo y jefatura. El dump de referencia del 24-08-2026 todavía sólo contemplaba los 3 roles originales — las empresas creadas antes de este cambio necesitaron un backfill de datos para sembrar los 2 roles nuevos, incluido en la migración de la base de datos (cambio N°5).

Deuda técnica derivada, en resolución: dos sistemas de control de acceso conviviendo en el código (uno centralizado en lib/auth.php, otro propio del módulo de Usuarios) que no reconocen los mismos roles entre sí. Ver seguimiento aparte de deuda técnica de Gestión de Usuarios/Empresas.

"La deuda técnica derivada fue resuelta el 08-09-2026 mediante la centralización del control de acceso en lib/auth.php y la definición de comportamiento para los cinco roles. La validación funcional definitiva de la matriz de permisos con el Cliente queda incorporada como actividad de Etapa 2."

5. Migración de servidor y base de datos

Fecha: 28-08-2026.

La base de datos se migró de tecaivot_SCT (servidor 201.148.105.87) a safetyco_SCT (servidor 201.148.104.98). La migración se probó de punta a punta antes de ejecutarse en producción, e incluyó el backfill de roles del cambio N°4.

6. Separación de marca: Tecaivot y SCT en sitios independientes

Fecha: 28-08-2026.

www.tecaivot.cl pasó a mostrar únicamente información de la empresa, con un enlace hacia Safety Control Tower para quien busque el producto. El sitio de SCT (www.safetycontoltower.cl) se reenfocó 100% en el producto, quitando referencias a Tecaivot salvo la mención de copyright y desarrollo en el pie de página ("© Tecaivot — Desarrollado por Helheim.cl").

Como parte de este cambio, SCT recibió una identidad visual propia: nuevo isotipo, favicon y paleta de colores (navy, celeste y dorado), reemplazando la paleta que hasta entonces compartía con Tecaivot.

7. Tabla nueva no contemplada en el diccionario original: security_event_evidence

Fecha: 28-08-2026.

El alcance de "Registro de eventos/incidentes" exige explícitamente "carga de evidencias (fotografías y documentos)", pero el diccionario de base de datos exportado el 24-08-2026 no contemplaba ninguna tabla para este propósito. Se agregó la tabla security_event_evidence mediante una migración aditiva y separada (no se modificó la migración original ya aplicada), documentada en el Anexo actualizado.

8. Cierre técnico de Gestión de Usuarios y Empresa

Fecha: 08-09-2026.

Ya quedaron implementados: frontend completo de Usuarios/Empresas, lib/auth.php centralizado, rol jefatura, respuesta JSON unificada, baja/reactivación lógica de empresas, validación real de RUT, eliminación del sistema paralelo de autorización, interfaz multilenguaje, fotografía de usuario, logotipo de empresa, cache-busting y funcionamiento en localhost/desarrollo web.

Etapas del servicio

Eta 1 – Desarrollo MVP (FP 28 agosto)

Objetivo etapa 1: Diseñar, desarrollar e implementar un Producto Mínimo Viable (MVP) funcional del sistema Safety Control Tower (SCT)

El MVP estará orientado a habilitar la operación base del sistema, permitiendo validar el modelo con usuarios reales.

Estado real 08-09-2026: funcionalmente completa

- **Credenciales de acceso al hosting www.tecaivot.cl**
 - Credenciales FTP (o SFTP)
 - IP del Servidor: 201.148.105.87
 - Usuario: tecaivot
 - Password: 81V+hwXUb6.6Gz
 - Configuración ambiente de **desarrollo**
 - home/tecaivot/public_html/dev
 - <http://www.tecaivot.cl/dev/>
 - Configuración ambiente de **producción**
 - home/tecaivot/public_html/
 - <http://www.tecaivot.cl>
 - Credenciales cPanel
 - <http://201.148.105.87/cpanel>
 - Usuario: tecaivot
 - Password: 81V+hwXUb6.6Gz
 - database “tecaivot_SCT”
- **Credenciales de acceso al hosting www.safetycontrolltower.cl**
 - Credenciales FTP (o SFTP)
 - IP del Servidor: 201.148.104.98
 - Usuario: safetyco
 - Password: cBz1t89IJ6*Y+B
 - Configuración ambiente de **desarrollo**
 - /public_html/dev
 - www.safetycontrolltower.cl/dev
 - Configuración ambiente de **producción**
 - home/public_html
 - www.safetycontrolltower.cl
 - Credenciales cPanel
 - <https://www.safetycontrolltower.cl:2083/>
 - Usuario: safetyco
 - Password: cBz1t89IJ6*Y+B
 - database “safetyco_SCT”
- **Manual de Marca**
 - [Manual de Marca V1](#)
 - [Carpeta de imágenes del manual de marca](#)

- Infraestructura y arquitectura base

Módulo / Actividad	Estado	Responsable
Hosting, dominio y despliegue	RESUELTO	Helheim
Migración a servidor y base de datos nuevos (safetyco_SCT)	RESUELTO <i>ver cambio de alcance N°5</i>	Helheim
SSL/HTTPS en ambos dominios	RESUELTO	Helheim
Arquitectura PHP nativo (PDO, CSRF, rate limiting, helper de respuesta único)	RESUELTO	Helheim
Base de datos: modelo de datos, 33 tablas (32 + evidencias)	RESUELTO	Helheim
Manual de Marca	RESUELTO	Cliente
SEO (técnico, on-page, contenido, local)	RESUELTO <i>técnico completo; SEO local pendiente de dirección física real de la empresa</i>	Cliente local - Trasladado a Etapa 2; depende de información física del Cliente

- **Módulos funcionales**

Módulo / Actividad	Estado	Responsable
Autenticación(login sin contraseña, código por correo)	RESUELTO	Helheim
Gestión de Empresas	RESUELTO	Helheim
Gestión de Usuarios	RESUELTO	Helheim
Proyectos	RESUELTO	Helheim
Centros / Sedes	RESUELTO	Helheim
Trabajadores (perfil, foto)	RESUELTO	Helheim
Inducción (banco de preguntas, evaluación, certificado PDF)	RESUELTO	Helheim
Eventos / Incidentes (seguimiento, evidencias)	RESUELTO	Helheim
Dashboard básico con indicadores	RESUELTO	Helheim
Sitio público multi-idioma (5 idiomas)	RESUELTO	Helheim

- Notas de cierre de Etapa 1
 - Dos actividades de infraestructura quedaron explícitamente pospuestas hasta el cierre del proyecto en Etapa 3, por decisión del CTO y Sponsor: enviar y verificar los sitemaps de ambos dominios en Google Search Console y Bing Webmaster Tools (la verificación de dominio de safetycontrolltower.cl en Google Search Console sí se completó)
- Convenciones de arquitectura PHP Nativo, sin uso de Laravel.
 - PDO con sentencias preparadas en todo — nunca SQL concatenado con datos del usuario.
 - CSRF token en cada formulario que modifica datos.
 - password_hash/password_verify para todo dato sensible que se compara (código OTP incluido).
 - Rate limiting con tabla dedicada (login_attempts, login_codes) en vez de lógica ad-hoc.
 - Aislamiento multiempresa vía id_company presente en prácticamente todas las tablas.
 - Migraciones SQL versionadas y probadas contra una base de datos real antes de aplicarse.
 - config.php separa la conexión a BD de la lógica de negocio, con credenciales por variable de entorno.
 - Manejo de errores: el detalle real se registra con error_log() y al cliente solo se devuelve un mensaje genérico — nunca se expone la excepción.
 - Estructura de carpetas por módulo
 - /api/empresas/, /api/trabajadores/, /api/eventos/...
 - un archivo por acción (listar.php, crear.php, editar.php).
 - /lib/ — código compartido: conexión a BD, autenticación, helpers de respuesta, validaciones.
 - /partials/ y /lang/ - ya existen y funcionan bien; se mantienen igual.
 - Helper de respuesta JSON Único
 - Con varios módulos nuevos en camino, eso es una fuente segura de inconsistencia (alguno olvida el código HTTP, otro el campo success). Un solo helper reutilizable - algo como responder JSON (\$exito, \$datos, \$mensaje, \$codigoHttp) - evita que cada desarrollador reinvente el formato de respuesta.
 - Control de acceso por rol centralizado
 - La tabla users_role/users_role_group ya existe, pero ningún endpoint construido hasta ahora la usa para restringir acceso. Antes de construir empresas/trabajadores/eventos, conviene definir una función única (ej. requireRole(['administrador','cliente'])) que se llame al inicio de cada endpoint protegido, en vez de que cada archivo nuevo reimplemente su propia validación de permisos.
 - Capa simple de acceso a datos por unidad
 - Agrupar las consultas de cada entidad en un solo lugar (ej. lib/repositorios/EmpresaRepository.php) reduce ese riesgo y facilita corregir una consulta en un solo sitio si cambia el modelo de datos.
 - Convención de nombres de columna
 - El diccionario de base de datos (Anexo) ya muestra columnas inconsistentes heredadas: created_by en unas tablas, create_by en otras; date_create en unas, date_created en otras. No conviene corregir retroactivamente por el riesgo de romper código en uso, pero sí fijar created_by / date_create / last_update como el estándar único para toda tabla nueva de aquí en adelante.
 - Checklist min de prueba por determinar
 - Versionado de ambiente de desarrollo
 - La convención vs1, vs2, vs3... que ya se usa en /dev/tecaivot_SCT_E1/ funciona bien y se mantendrá formalmente como estándar del proyecto para cada entrega, incluyendo Etapa 2 y 3.

Deuda técnica de Gestión de Usuarios y Empresas - CERRADA por JACL

- js/usuarios.js y js/empresas.js disponibles y operativos.
- Control de acceso unificado en lib/auth.php.
- Roles administrador_completo y jefatura reconocidos en módulos correspondientes.
- Política funcional de jefatura implementada.
- Respuesta JSON unificada.
- Baja/reactivación lógica de empresas implementada.
- Validación real de RUT conectada a creación/edición.
- Código muerto de autorización paralela eliminado.
- Soporte ES/EN/PT/FR/ZH en Usuarios y Empresas.
- Fotografía de usuario y logotipo de empresa implementados.

Actividades heredadas trasladadas a Etapa 2

- Validación funcional de la matriz completa de 5 roles con el Cliente.
- Documentación de autenticación contraseña + OTP.
- Actualización formal del diccionario de base de datos y política de roles.
- Cierre del Manual de Marca.
- SEO local sujeto a información física del Cliente.

Etapa 2 – Desarrollo Core Funcional (FP 30 de Septiembre)

Objetivo etapa 2: Completar los módulos críticos del negocio, consolidando la operación del sistema.

Estado real: En curso

Módulo / Actividad	Estado	Responsable
Auditorías	RESUELTO reutiliza el motor de evaluación ya construido en Inducción. Desarrollo conjunto con Autoevaluaciones sobre company_test; administración, asignación, ejecución, resultados y trazabilidad.	Helheim
Self-assessment	RESUELTO Mismo motor; type=autoevaluacion, con flujo y permisos propios.	Helheim
Protocolos MINSAL parametrizables	RESUELTO Tabla base existe; se integrará con formularios dinámicos.	Helheim
Motor de formularios dinámicos	RESUELTO Tablas base existen; falta lógica de aplicación y persistencia de respuestas.	Helheim
Gestión documental / descarga de certificados	RESUELTO Completado en Inducción durante Etapa 1.	Helheim
Dashboard avanzado (tendencias, filtros más finos)	RESUELTO Extiende el dashboard básico ya construido. Tendencias y filtros sobre datos consolidados de Etapa 2.	Helheim
Mejoras UX/UI	EN CURSO Consolidación visual y responsive al cierre funcional de Etapa 2	Helheim
ConE1 - Validación Matriz de 5 roles	RESUELTO	Helheim/sponsor
ConE1 - Documentación autenticación contraseña + OTP	EN CURSO	Helheim
ConE1 - Actualización diccionario BD y política de roles	EN CURSO	Helheim
ConE1 - Manual de marca SCT/Tecaivot	EN CURSO	Helheim
ConE1 - SEO Local	EN CURSO	Helheim

Etapa 3 – Optimización y Escalabilidad (FP 30 de Octubre)

Objetivo etapa 3: Optimizar el sistema, fortalecer su trazabilidad y habilitar escalabilidad modular.

Estado real: En curso, adelantando desarrollos de módulos

Módulo / Actividad	Estado	Responsable
Seguimiento de programas (tracking mensual)	RESUELTO	Helheim
KPIs agregados y consolidación de indicadores	EN CURSO	Helheim
Historial de cambios (auditoría a nivel de campo)	RESUELTO	Helheim
Control de acceso avanzado (roles y permisos granulares)	RESUELTO	Helheim
Módulos adicionales: Medio ambiente, Arqueología	PENDIENTE	Helheim
Optimización de rendimiento	PENDIENTE	Helheim
Testing (QA)	PENDIENTE	Helheim
Hardening de seguridad	PENDIENTE	Helheim
Documentación técnica	EN CURSO	Helheim
Documentación de usuario	EN CURSO	Helheim
Despliegue final y capacitación a usuarios administradores del Cliente	PENDIENTE	Helheim
Enviar sitemap de safetycontrolltower.cl en Google Search Console	EN CURSO <i>dominio ya verificado</i>	Helheim
Verificar tecaivot.cl en Google Search Console y enviar su sitemap	PENDIENTE	Cliente/Helheim
Verificación + sitemap de ambos dominios en Bing Webmaster Tools	PENDIENTE	Cliente / Helheim

Anexo Diccionario de base de datos - SAFETY CONTROL TOWER (SCT)

- state: 1 = activo, 0 = inactivo, salvo indicación específica.
- date_create / created_at y last_update se completan mediante la aplicación o valores por defecto según la tabla.
- id_company implementa aislamiento lógico multiempresa.
- Las estructuras correspondientes a Etapa 2 y Etapa 3 fueron creadas anticipadamente.
- La existencia de una tabla no implica que el módulo funcional correspondiente esté terminado.
- Las contraseñas nunca se almacenan en texto plano.
- El RUT no se utiliza como contraseña ni participa en la generación de password_hash.

ÍNDICE — 34 tablas documentadas

Etapa 1			Etapa 2			Etapa 3		
users			company_test			permissions		
user_credentials			audits			role_permissions		
login_codes			certificates			event type		
login_attempts			dynamic_forms			security_events		
password_resets			dynamic_forms_fields			security_event_tracking		
log			protocols			security_event_evidence		
company			programs			program_monthly_tracking		
company_center						change_history		
projects								
workers								
worker_projects								
users_role_group								
users_role								
questions								
questions_options								
company_test_rel_questions								
users_test_assigned								
users_test_answers								
test_materials								

1. AUTENTICACIÓN Y TRAZABILIDAD DE USUARIOS

TABLA: users

Etapa: 1

Uso:

Almacena la identidad y datos funcionales de las cuentas de usuario.

La contraseña no se almacena en users.

Las credenciales de contraseña están separadas en user_credentials.

Campos:

id_users

Tipo: varchar(50)

Regla: PRIMARY KEY

Descripción:

Correo electrónico del usuario e identificador utilizado para iniciar sesión.

id_company

Tipo: int

Regla: FOREIGN KEY

Descripción:

Empresa a la que pertenece la cuenta.

id_worker

Tipo: int

Regla: FOREIGN KEY, admite NULL

Descripción:

Ficha de trabajador asociada cuando corresponde.

name

Tipo: varchar(50)

Descripción:

Nombre del usuario.

lastname

Tipo: varchar(50)

Descripción:

Apellido del usuario.

rut

Tipo: varchar(10)

Descripción:

RUT chileno normalizado y validado.

state

Tipo: int

Descripción:

1 = activo.

0 = inactivo.

language

Tipo: varchar(11)

Descripción:

Idioma preferido del usuario.

Valores soportados:

es

en

pt

fr

zh

profile_photo_path

Tipo: varchar(255)

Admite NULL: Sí

Descripción:

Ruta relativa de la fotografía o avatar del usuario.

last_access

Tipo: datetime

Descripción:

Fecha y hora del último acceso exitoso.

created_by

Tipo: varchar(50)

Descripción:

Usuario o proceso que creó la cuenta.

date_create

Tipo: datetime

Descripción:

Fecha de creación.

last_update

Tipo: datetime

Descripción:

Fecha de última actualización.

Llaves foráneas:

id_company -> company.id_company

id_worker -> workers.id_worker

TABLA: user_credentials

Etapa: 1

Uso:

Almacenamiento exclusivo de las credenciales de contraseña.

Separa los datos de autenticación de los datos personales almacenados en users.

Una cuenta puede no tener una credencial activa hasta completar su proceso de activación o restablecimiento.

Campos:

id_users

Tipo: varchar(50)

Regla: PRIMARY KEY, FOREIGN KEY

Descripción:

Cuenta asociada.

password_hash

Tipo: varchar(255)

Admite NULL: Sí

Descripción:

Hash de la contraseña generado mediante password_hash() utilizando PASSWORD_DEFAULT.

Nunca contiene:

- RUT
- contraseña en texto plano
- contraseña temporal reversible

credential_status

Tipo: enum

Valores:

pending_activation

reset_required

active

Descripción:

pending_activation:

El usuario todavía debe crear su primera contraseña.

reset_required:

La contraseña histórica fue invalidada y debe establecerse una nueva.

active:

Existe una contraseña válida configurada.

password_changed_at

Tipo: datetime

Admite NULL: Sí

Descripción:

Fecha del último cambio de contraseña.

legacy_password_invalidated_at

Tipo: datetime

Admite NULL: Sí

Descripción:

Fecha en que se invalidó una credencial histórica.

created_at

Tipo: datetime

Descripción:

Fecha de creación del registro.

updated_at

Tipo: datetime

Descripción:

Fecha de última actualización.

Llave foránea:

id_users -> users.id_users

TABLA: login_codes

Etapa: 1

Uso:

Almacena los códigos OTP correspondientes al segundo factor de autenticación.

El OTP solamente se genera después de validar correctamente correo + contraseña.

El código:

- tiene 6 dígitos
- se almacena hasheado
- expira
- es de un solo uso

Campos:

id_login_code

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

Descripción:

Identificador del código OTP.

id_users

Tipo: varchar(50)

Regla: FOREIGN KEY

Descripción:

Usuario al que pertenece el OTP.

code_hash

Tipo: varchar(255)

Descripción:

Hash del código OTP.

expires_at

Tipo: datetime

Descripción:

Fecha y hora de vencimiento.

Vigencia actual:

10 minutos.

used_at

Tipo: datetime

Admite NULL: Sí

Descripción:

Fecha en que fue utilizado correctamente.

NULL significa que aún no ha sido utilizado.

attempts

Tipo: int
Descripción:
Cantidad de intentos fallidos de validación contra el OTP.

ip_address
Tipo: varchar(45)
Admite NULL: Sí
Descripción:
Dirección IP desde la que fue solicitado.

created_at
Tipo: datetime
Descripción:
Fecha de creación.

Llave foránea:

id_users -> users.id_users

TABLA: login_attempts

Etapa: 1

Uso:

Registro de intentos del primer factor de autenticación:

correo + contraseña

También se utiliza para controlar el bloqueo temporal por intentos fallidos.

Campos:

id

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

Descripción:

Identificador del intento.

identifier

Tipo: varchar(255)

Descripción:

Correo utilizado en el intento.

No posee FK deliberadamente para permitir registrar intentos contra correos inexistentes sin revelar si una cuenta está registrada.

ip_address

Tipo: varchar(45)

Descripción:

IP desde la que se realizó el intento.

success

Tipo: tinyint(1)

Descripción:

1 = login de primer factor correcto.

0 = intento fallido.

Los registros success = 1 se conservan para auditoría, pero NO participan en el cálculo del bloqueo.

created_at

Tipo: datetime

Descripción:

Fecha y hora del intento.

Política de bloqueo:

5 intentos fallidos de contraseña
dentro de 10 minutos

=

bloqueo durante 15 minutos.

Los accesos correctos no aumentan ni prolongan el bloqueo.

El OTP posee su propio control de intentos mediante login_codes.

TABLA: password_resets

Etapa: 1

Estado: ACTIVA

Uso:

Gestiona:

- activación inicial de contraseña
- restablecimiento de contraseña

Utiliza tokens aleatorios de un solo uso.

El token original nunca se almacena en texto plano en la base de datos.

Campos:

id_reset

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

Descripción:

Identificador del proceso.

id_users

Tipo: varchar(50)

Regla: FOREIGN KEY

Descripción:

Usuario afectado.

token_hash

Tipo: varchar(255)

Descripción:

Hash SHA-256 del token aleatorio.

El token original utiliza 256 bits de entropía.

purpose

Tipo: enum

Valores:

activation

reset

Descripción:

activation:

Configuración inicial de contraseña.

reset:

Restablecimiento de contraseña existente.

expires_at

Tipo: datetime

Descripción:
Vencimiento del token.

used_at
Tipo: datetime
Admite NULL: Sí
Descripción:
Fecha en que el token fue utilizado.

ip_address
Tipo: varchar(45)
Admite NULL: Sí
Descripción:
IP desde la que se solicitó.

created_at
Tipo: datetime
Descripción:
Fecha de creación.

Llave foránea:

id_users -> users.id_users

TABLA: log
Etapa: 1

Uso:
Bitácora general de acciones realizadas por usuarios para mantener trazabilidad.

Campos:

id_log
Tipo: int
Regla: PRIMARY KEY, AUTO_INCREMENT
Descripción:
Identificador del registro.

id_users
Tipo: varchar(50)
Regla: FOREIGN KEY
Descripción:
Usuario que realizó la acción.

id_action
Tipo: int
Descripción:
Código interno correspondiente al tipo de acción.

bd_msj
Tipo: varchar(50)
Descripción:
Detalle breve asociado a la acción.

create_date
Tipo: datetime
Descripción:
Fecha y hora del registro.

Llave foránea:

id_users -> users.id_users

FLUJO DE AUTENTICACIÓN VIGENTE

1. El usuario ingresa correo electrónico y contraseña.
2. El sistema valida:
 - estado de cuenta
 - credential_status = active
 - password_verify()
3. Si la contraseña es correcta:
 - genera un OTP

- almacena su hash
- envía el OTP por correo

4. El usuario ingresa el OTP dentro de su vigencia.

5. El OTP se valida.

6. Se marca el OTP como utilizado.

7. Se regenera el identificador de sesión.

8. El usuario accede al sistema.

Activación y restablecimiento utilizan:

password_resets

user_credentials

Regla:

El RUT es exclusivamente un dato de identidad.

No participa en la generación ni validación de contraseñas.

=====

2. ESTRUCTURA ORGANIZACIONAL

EMPRESAS, CENTROS, PROYECTOS Y TRABAJADORES

=====

TABLA: company

Etapa: 1

Uso:

Empresas que utilizan Safety Control Tower.

La eliminación funcional es lógica mediante state = 0.

Una empresa puede ser posteriormente reactivada.

Campos:

id_company

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

Descripción:

Identificador de empresa.

rut

Tipo: varchar(50)

Regla: UNIQUE

Descripción:

RUT de la empresa.

razon_social

Tipo: varchar(150)

Descripción:

Razón social.

address

Tipo: varchar(255)

Descripción:

Dirección.

email

Tipo: varchar(50)

Descripción:

Correo de contacto.

logo_path

Tipo: varchar(255)

Admite NULL: Sí

Descripción:

Ruta relativa del logotipo corporativo.

state

Tipo: int

Descripción:

1 = activa.

0 = inactiva.

created_by

Tipo: varchar(50)

Descripción:

Usuario o proceso creador.

date_create

Tipo: datetime

Descripción:

Fecha de creación.

last_update

Tipo: datetime

Descripción:

Última actualización.

Restricción de unicidad:

rut

Funcionalidades implementadas:

- creación
- consulta
- modificación
- baja lógica
- reactivación
- carga de logotipo
- eliminación de logotipo
- interfaz multilenguaje

TABLA: company_center

Etapa: 1

Uso:

Centros, sedes o faenas pertenecientes a una empresa.

Campos:

id_company_center

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_company

Tipo: int

Regla: FOREIGN KEY

Descripción:

Empresa dueña del centro.

name

Tipo: varchar(50)

Descripción:

Nombre del centro o sede.

description

Tipo: varchar(255)

Descripción:

Descripción.

state

Tipo: int

Descripción:

1 = activo.

0 = inactivo.

create_by

Tipo: varchar(50)

Descripción:

Usuario o proceso creador.

date_create

Tipo: datetime

Descripción:

Fecha de creación.

last_update

Tipo: datetime

Descripción:

Última actualización.

Llave foránea:

id_company -> company.id_company

ON DELETE NO ACTION
ON UPDATE CASCADE

TABLA: projects

Etapa: 1

Uso:

Proyectos u obras pertenecientes a una empresa.

Se utiliza para filtros, eventos, indicadores y dashboard por empresa/proyecto.

Campos:

id_project

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_company

Tipo: int

Regla: FOREIGN KEY

Descripción:

Empresa propietaria.

name

Tipo: varchar(150)

Descripción:

Nombre del proyecto.

description

Tipo: varchar(255)

Admite NULL: Sí

Descripción:

Descripción.

state

Tipo: int

Descripción:

1 = activo.

0 = inactivo.

created_by

Tipo: varchar(50)

Descripción:

Usuario/proceso creador.

date_create

Tipo: datetime

last_update

Tipo: datetime

Llave foránea:

id_company -> company.id_company

ON DELETE NO ACTION
ON UPDATE CASCADE

TABLA: workers

Etapa: 1

Uso:

Ficha personal y laboral de trabajadores.

Un trabajador no necesariamente posee una cuenta de usuario.

Campos:

id_worker

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_company

Tipo: int

Regla: FOREIGN KEY

Descripción:

Empresa del trabajador.

rut

Tipo: varchar(12)

Descripción:

RUT del trabajador.

name

Tipo: varchar(100)

Descripción:

Nombre.

lastname

Tipo: varchar(100)

Descripción:

Apellido.

email

Tipo: varchar(150)

Admite NULL: Sí

Descripción:

Correo.

phone

Tipo: varchar(20)

Admite NULL: Sí

Descripción:

Teléfono.

position

Tipo: varchar(100)

Admite NULL: Sí

Descripción:

Cargo.

photo_path
Tipo: varchar(255)
Admite NULL: Sí
Descripción:
Fotografía del trabajador.

state
Tipo: int
Descripción:
1 = activo.
0 = inactivo.

created_by
Tipo: varchar(50)

date_create
Tipo: datetime

last_update
Tipo: datetime

Llave foránea:

id_company -> company.id_company
ON DELETE NO ACTION
ON UPDATE CASCADE

Restricción de unicidad:

uq_worker_rut_company
(rut, id_company)

TABLA: worker_projects
Etapa: 1

Uso:
Tabla puente entre trabajadores y proyectos.

Permite relación muchos a muchos.

Campos:

id_worker
Tipo: int
Regla: PRIMARY KEY COMPUESTA, FOREIGN KEY

id_project
Tipo: int
Regla: PRIMARY KEY COMPUESTA, FOREIGN KEY

date_create
Tipo: datetime
Descripción:
Fecha de asignación.

Llaves foráneas:

id_project -> projects.id_project
ON DELETE CASCADE
ON UPDATE CASCADE

id_worker -> workers.id_worker
ON DELETE CASCADE
ON UPDATE CASCADE

=====

3. ROLES Y PERMISOS

=====

TABLA: users_role_group

Etapas: 1

Uso:

Catálogo de roles disponibles por empresa.

Actualmente Safety Control Tower utiliza cinco roles funcionales.

Roles:

administrador_completo

Descripción:

Administración global y multiempresa.

administrador

Descripción:

Administración de la empresa según el contexto autorizado.

cliente

Descripción:

Representante de empresa cliente con acceso a información y operaciones habilitadas.

jefatura

Descripción:

Gestión operativa de su empresa.

Puede gestionar trabajadores y crear/asignar cuentas trabajador, sin escalar privilegios.

trabajador

Descripción:

Acceso operativo a funciones asignadas y administración de su perfil.

Campos:

id_role_group

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_company

Tipo: int

Regla: FOREIGN KEY

name

Tipo: varchar(50)

description
Tipo: varchar(50)

state
Tipo: int

create_by
Tipo: varchar(50)

date_create
Tipo: datetime

last_update
Tipo: datetime

Llave foránea:

id_company -> company.id_company
ON DELETE NO ACTION
ON UPDATE CASCADE

TABLA: users_role

Etapas: 1

Uso:

Asigna uno o más roles a una cuenta de usuario.

El control de acceso de la aplicación está centralizado en:

lib/auth.php

Campos:

id_users_role

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_users

Tipo: varchar(50)

Regla: FOREIGN KEY

id_role_group

Tipo: int

Regla: FOREIGN KEY

state

Tipo: int

create_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

Llaves foráneas:

id_role_group -> users_role_group.id_role_group

ON DELETE CASCADE

ON UPDATE CASCADE

id_users -> users.id_users

ON DELETE CASCADE

ON UPDATE CASCADE

TABLA: permissions
Etapa: 3
Estado: estructura creada, funcionalidad pendiente

Uso:
Catálogo de permisos granulares para el control de acceso avanzado.

Ejemplos:

workers.create
events.view

Campos:

id_permission
Tipo: int
Regla: PRIMARY KEY, AUTO_INCREMENT

code
Tipo: varchar(80)
Regla: UNIQUE
Descripción:
Código único del permiso.

description
Tipo: varchar(255)
Admite NULL: Sí

Restricción:

uq_permission_code
(code)

TABLA: role_permissions

Etapas: 3

Estado: estructura creada, funcionalidad pendiente

Uso:

Tabla puente entre roles y permisos.

Campos:

id_role_group

Tipo: int

Regla: PRIMARY KEY COMPUESTA, FOREIGN KEY

id_permission

Tipo: int

Regla: PRIMARY KEY COMPUESTA, FOREIGN KEY

Llaves foráneas:

id_role_group -> users_role_group.id_role_group

ON DELETE CASCADE

ON UPDATE CASCADE

id_permission -> permissions.id_permission

ON DELETE CASCADE

ON UPDATE CASCADE

=====

4. EVALUACIONES, INDUCCIÓN Y CERTIFICACIÓN

=====

TABLA: company_test

Etapa: 1 / 2

Uso:

Motor genérico de evaluaciones.

Se utiliza para:

- inducciones
- autoevaluaciones
- auditorías
- otros tipos futuros

Campos:

id_test

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

name

Tipo: varchar(50)

Descripción:

Nombre de la evaluación.

type

Tipo: enum

Valores:

inducccion

autoevaluacion

auditoria

otro

description

Tipo: varchar(255)

Descripción:

Descripción.

version

Tipo: int

Descripción:

Versión del contenido.

state

Tipo: int

Descripción:

1 = activo.
0 = inactivo.

attempts_allowed

Tipo: int

Descripción:

Cantidad de intentos permitidos.

approval_percentage

Tipo: int

Descripción:

Porcentaje mínimo requerido para aprobación.

effective_date_from

Tipo: datetime

Descripción:

Inicio de vigencia.

effective_date_until

Tipo: datetime

Descripción:

Fin de vigencia.

created_by

Tipo: varchar(50)

date_created

Tipo: datetime

last_update

Tipo: datetime

id_company

Tipo: int

Regla: FOREIGN KEY

Descripción:

Empresa propietaria de la evaluación.

Llave foránea:

id_company -> company.id_company

ON DELETE NO ACTION

ON UPDATE CASCADE

TABLA: questions

Etapas: 1

Uso:

Banco de preguntas utilizado por company_test.

Campos:

id_questions

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

question

Tipo: text

Descripción:

Texto de la pregunta.

url_add_material

Tipo: varchar(50)

Descripción:

Enlace opcional a material de apoyo.

difficulty

Tipo: int

Descripción:

Nivel de dificultad.

points

Tipo: int

Descripción:

Puntaje de la pregunta.

state

Tipo: int

add_expl_question

Tipo: varchar(100)

Descripción:

Explicación adicional de la pregunta.

create_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

TABLA: questions_options

Etapas: 1

Uso:

Alternativas posibles para las preguntas.

Campos:

id_questions_options

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_questions

Tipo: int

Regla: FOREIGN KEY

text_option

Tipo: varchar(50)

Descripción:

Texto de la alternativa.

is_it_co

Tipo: int

Descripción:

Indica si corresponde a la respuesta correcta.

add_expl_opt

Tipo: varchar(100)

Descripción:

Explicación adicional de la alternativa.

state

Tipo: int

created_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

Llave foránea:

id_questions -> questions.id_questions

ON DELETE CASCADE

ON UPDATE CASCADE

TABLA: company_test_rel_questions
Etapa: 1

Uso:
Relaciona evaluaciones con preguntas.

También permite definir el puntaje específico que tendrá una pregunta dentro de una evaluación determinada.

Campos:

id_rel
Tipo: int
Regla: PRIMARY KEY, AUTO_INCREMENT

id_test
Tipo: int
Regla: FOREIGN KEY

id_question
Tipo: int
Regla: FOREIGN KEY

assigned_score
Tipo: int
Descripción:
Puntaje asignado a la pregunta dentro de esa evaluación.

created_by
Tipo: varchar(50)

date_create
Tipo: datetime

last_update
Tipo: datetime

Llaves foráneas:

id_question -> questions.id_questions
ON DELETE CASCADE
ON UPDATE CASCADE

id_test -> company_test.id_test
ON DELETE CASCADE
ON UPDATE CASCADE

TABLA: users_test_assigned

Etapas: 1

Uso:

Asignación de evaluaciones a usuarios.

Campos:

id_user_test_assigned

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_users

Tipo: varchar(50)

Regla: FOREIGN KEY

id_test

Tipo: int

Regla: FOREIGN KEY

id_company

Tipo: int

Regla: FOREIGN KEY

asignamente_date

Tipo: datetime

Descripción:

Fecha de asignación.

deadline

Tipo: datetime

Descripción:

Fecha máxima para completar la evaluación.

state

Tipo: int

created_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

Llaves foráneas:

id_company -> company.id_company

ON DELETE NO ACTION

ON UPDATE CASCADE

id_test -> company_test.id_test
ON DELETE NO ACTION
ON UPDATE CASCADE

id_users -> users.id_users
ON DELETE NO ACTION
ON UPDATE CASCADE

TABLA: users_test_answers

Etapas: 1

Uso:

Almacena las respuestas realizadas por usuarios durante cada intento de una evaluación.

Campos:

id_users_test_answers

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_users

Tipo: varchar(50)

Regla: FOREIGN KEY

id_company

Tipo: int

Regla: FOREIGN KEY

id_test

Tipo: int

Regla: FOREIGN KEY

id_test_try

Tipo: int

Descripción:

Número de intento.

id_rel

Tipo: int

Regla: FOREIGN KEY

Descripción:

Relación pregunta-evaluación.

id_question

Tipo: int

Regla: FOREIGN KEY

id_questions_options

Tipo: int

Regla: FOREIGN KEY

Descripción:

Alternativa seleccionada.

date_create

Tipo: datetime

last_update

Tipo: datetime

Llaves foráneas:

id_company -> company.id_company

id_questions_options -> questions_options.id_questions_options

id_question -> questions.id_questions

id_rel -> company_test_rel_questions.id_rel

id_test -> company_test.id_test

id_users -> users.id_users

Todas con ON DELETE NO ACTION y ON UPDATE CASCADE según el modelo actual.

TABLA: test_materials

Etapas: 1

Uso:

Material complementario asociado a una evaluación o curso.

Campos:

id_material

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_test

Tipo: int

Regla: FOREIGN KEY

title

Tipo: varchar(150)

material_type

Tipo: enum

Valores:

documento

texto

video

otro

file_path

Tipo: varchar(255)

Admite NULL: Sí

content_text

Tipo: text

Admite NULL: Sí

sort_order

Tipo: int

created_by

Tipo: varchar(50)

date_create

Tipo: datetime

Llave foránea:

id_test -> company_test.id_test

ON DELETE CASCADE

ON UPDATE CASCADE

TABLA: certificates

Etapas: 1

Uso:

Certificados PDF generados cuando un usuario aprueba una evaluación de inducción.

Campos:

id_certificate

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_user_test_assigned

Tipo: int

Regla: FOREIGN KEY

code

Tipo: varchar(50)

Regla: UNIQUE

Descripción:

Código único de verificación.

file_path

Tipo: varchar(255)

Descripción:

Ruta del PDF.

issued_at

Tipo: datetime

state

Tipo: int

created_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

Llave foránea:

id_user_test_assigned ->

users_test_assigned.id_user_test_assigned

ON DELETE NO ACTION

ON UPDATE CASCADE

Restricción:

uq_certificate_code
(code)

=====

5. EVENTOS Y SEGURIDAD OPERACIONAL

=====

TABLA: event_types

Etapa: 1 / 3

Uso:

Catálogo de tipos de eventos o incidentes.

La misma estructura puede servir posteriormente para:

- seguridad
- medio ambiente
- arqueología

Campos:

id_event_type

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

module

Tipo: varchar(30)

Valores funcionales:

seguridad

medio_ambiente

arqueologia

name

Tipo: varchar(100)

description

Tipo: varchar(255)

Admite NULL: Sí

state

Tipo: int

TABLA: security_events

Etapas: 1 / 3

Uso:

Registro de eventos e incidentes.

Diseñada para ser reutilizada por futuros módulos de Medio Ambiente y Arqueología.

Campos:

id_security_events

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_company

Tipo: int

Regla: FOREIGN KEY

Descripción:

Empresa donde ocurrió el evento.

id_company_center

Tipo: int

Regla: FOREIGN KEY

Descripción:

Centro o sede donde ocurrió.

id_project

Tipo: int

Regla: FOREIGN KEY, admite NULL

Descripción:

Proyecto asociado.

module

Tipo: varchar(30)

Descripción:

Módulo correspondiente.

Valores:

seguridad

medio_ambiente

arqueologia

id_worker

Tipo: int

Regla: FOREIGN KEY, admite NULL

Descripción:

Trabajador involucrado.

id_worker_name

Tipo: varchar(150)

Admite NULL: Sí

Descripción:

Nombre histórico del trabajador al momento del evento.

id_event

Tipo: int

Regla: FOREIGN KEY

Descripción:

Tipo de evento.

event_date

Tipo: datetime

Descripción:

Fecha y hora del evento.

description

Tipo: text

Descripción:

Descripción del evento.

criticality

Tipo: enum

Valores:

baja

media

alta

critica

state

Tipo: int

created_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

Llaves foráneas:

id_company_center -> company_center.id_company_center

ON DELETE NO ACTION

ON UPDATE CASCADE

id_company -> company.id_company

ON DELETE NO ACTION

ON UPDATE CASCADE

id_project -> projects.id_project

ON DELETE NO ACTION

ON UPDATE CASCADE

id_event -> event_types.id_event_type

ON DELETE NO ACTION

ON UPDATE CASCADE

id_worker -> workers.id_worker

ON DELETE NO ACTION

ON UPDATE CASCADE

TABLA: security_event_tracking

Etapas: 1

Uso:

Seguimiento de eventos e incidentes.

Permite registrar:

- avances
- compromisos
- responsables
- fechas
- plazos

Campos:

id_security_event_tracking

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_security_events

Tipo: int

Regla: FOREIGN KEY

tracking_description

Tipo: text

person_charge

Tipo: varchar(50)

commitment_date

Tipo: datetime

deadline

Tipo: datetime

created_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

Llave foránea:

id_security_events -> security_events.id_security_events

ON DELETE CASCADE

ON UPDATE CASCADE

TABLA: security_event_evidence

Etapas: 1

Uso:

Almacena fotografías y documentos asociados como evidencia de eventos o incidentes.

Campos:

id_evidence

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

Descripción:

Identificador de la evidencia.

id_security_events

Tipo: int

Regla: FOREIGN KEY

Descripción:

Evento al cual pertenece la evidencia.

file_path

Tipo: varchar(255)

Descripción:

Ruta del archivo almacenado.

original_name

Tipo: varchar(255)

Descripción:

Nombre original del archivo cargado.

file_type

Tipo: enum

Valores:

imagen

documento

Descripción:

Tipo de evidencia.

uploaded_by

Tipo: varchar(50)

Descripción:

Usuario que cargó el archivo.

date_create

Tipo: datetime

Descripción:

Fecha de carga.

Índice:

idx_evidence_event
(id_security_events)

Llave foránea:

id_security_events -> security_events.id_security_events
ON DELETE CASCADE
ON UPDATE CASCADE

=====

6. AUDITORÍAS

=====

TABLA: audits

Etapa: 2

Estado: desarrollo funcional EN CURSO

Uso:

Cabecera y trazabilidad específica de auditorías realizadas a una empresa.

No debe utilizarse como un segundo motor de evaluaciones.

Las preguntas, opciones, asignaciones, intentos y respuestas se gestionan mediante el motor común company_test.

Campos:

id_audits

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

Descripción:

Identificador de auditoría.

id_company

Tipo: int

Descripción:

Empresa auditada.

La FK formal hacia company debe incorporarse mediante migración de Etapa 2.

name_auditor

Tipo: varchar(50)

Descripción:

Nombre del auditor.

email

Tipo: varchar(50)

Descripción:

Correo del auditor.

score

Tipo: varchar(50)

Descripción:

Resultado o puntaje resumido.

obs

Tipo: text

Descripción:

Observaciones.

created_by
Tipo: varchar(50)

date_create
Tipo: datetime

last_update
Tipo: datetime

DECISIÓN DE DISEÑO ETAPA 2

Auditorías utiliza:

company_test.type = 'auditoria'

Self-assessment utiliza:

company_test.type = 'autoevaluacion'

Ambos comparten:

company_test
questions
questions_options
company_test_rel_questions
users_test_assigned
users_test_answers

audits conserva exclusivamente información y trazabilidad propia del proceso de auditoría.

=====

7. FORMULARIOS DINÁMICOS

=====

TABLA: dynamic_forms

Etapas: 2

Estado: estructura creada, funcionalidad pendiente

Uso:

Define formularios configurables por empresa.

Campos:

id_form

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_company

Tipo: int

Regla: FOREIGN KEY, admite NULL

Descripción:

Empresa propietaria.

NULL significa formulario genérico reutilizable.

name

Tipo: varchar(150)

description

Tipo: text

Admite NULL: Sí

state

Tipo: int

created_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

Llave foránea:

id_company -> company.id_company

ON DELETE NO ACTION

ON UPDATE CASCADE

TABLA: dynamic_form_fields

Etapas: 2

Estado: estructura creada, funcionalidad pendiente

Uso:

Define los campos que componen un formulario dinámico.

Campos:

id_field

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_form

Tipo: int

Regla: FOREIGN KEY

label

Tipo: varchar(150)

Descripción:

Etiqueta visible.

field_type

Tipo: varchar(30)

Tipos previstos:

text

number

date

select

checkbox

file

options

Tipo: text

Admite NULL: Sí

Descripción:

Opciones para select o checkbox, actualmente separadas mediante "|".

is_required

Tipo: tinyint(1)

Descripción:

1 = obligatorio.

0 = opcional.

sort_order

Tipo: int

Descripción:

Orden de presentación.

Llave foránea:

```
id_form -> dynamic_forms.id_form  
ON DELETE CASCADE  
ON UPDATE CASCADE
```

=====

8. PROTOCOLOS

=====

TABLA: protocols
Etapa: 2
Estado: estructura creada, funcionalidad pendiente

Uso:
Protocolos MINSAL parametrizables.

Campos:

id_protocol
Tipo: int
Regla: PRIMARY KEY, AUTO_INCREMENT

id_company
Tipo: int
Regla: FOREIGN KEY, admite NULL

Descripción:
Empresa propietaria.

NULL:
Protocolo base reutilizable por todas las empresas.

code
Tipo: varchar(50)
Descripción:
Código del protocolo.

name
Tipo: varchar(150)
Descripción:
Nombre.

description
Tipo: text
Admite NULL: Sí

parameters
Tipo: longtext
Admite NULL: Sí
Descripción:
Parametrización flexible almacenada en formato JSON.

state
Tipo: int

created_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

Llave foránea:

id_company -> company.id_company

ON DELETE NO ACTION

ON UPDATE CASCADE

Restricción de unicidad:

uq_protocol_code_company

(code, id_company)

=====

9. PROGRAMAS Y SEGUIMIENTO

=====

TABLA: programs

Etapas: 3

Estado: estructura creada, funcionalidad pendiente

Uso:

Programas asociados a una empresa o proyecto para seguimiento periódico.

Campos:

id_program

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

id_company

Tipo: int

Regla: FOREIGN KEY

id_project

Tipo: int

Regla: FOREIGN KEY, admite NULL

name

Tipo: varchar(150)

description

Tipo: text

Admite NULL: Sí

state

Tipo: int

created_by

Tipo: varchar(50)

date_create

Tipo: datetime

last_update

Tipo: datetime

Llaves foráneas:

id_company -> company.id_company

ON DELETE NO ACTION

ON UPDATE CASCADE

```
id_project -> projects.id_project  
ON DELETE NO ACTION  
ON UPDATE CASCADE
```

TABLA: program_monthly_tracking
Etapas: 3
Estado: estructura creada, funcionalidad pendiente

Uso:
Seguimiento mensual de avance de los programas.

Campos:

id_tracking
Tipo: int
Regla: PRIMARY KEY, AUTO_INCREMENT

id_program
Tipo: int
Regla: FOREIGN KEY

period_month
Tipo: date
Descripción:
Primer día del mes que se está informando.

progress_percentage
Tipo: decimal(5,2)
Descripción:
Porcentaje de avance.

comments
Tipo: text
Admite NULL: Sí

created_by
Tipo: varchar(50)

date_create
Tipo: datetime

last_update
Tipo: datetime

Llave foránea:

id_program -> programs.id_program
ON DELETE CASCADE
ON UPDATE CASCADE

Restricción:

uq_program_period
(id_program, period_month)

=====

10. HISTORIAL DE CAMBIOS

=====

TABLA: change_history

Etapa: 3

Estado: estructura creada, funcionalidad pendiente

Uso:

Auditoría genérica de modificaciones realizadas a campos de cualquier tabla del sistema.

Es distinta de log:

log:

registra acciones generales.

change_history:

registra exactamente qué dato cambió, su valor anterior y nuevo.

Campos:

id_change

Tipo: int

Regla: PRIMARY KEY, AUTO_INCREMENT

table_name

Tipo: varchar(64)

Descripción:

Tabla modificada.

record_id

Tipo: varchar(50)

Descripción:

Identificador del registro modificado.

field_name

Tipo: varchar(64)

Descripción:

Campo modificado.

old_value

Tipo: text

Admite NULL: Sí

Descripción:

Valor anterior.

new_value

Tipo: text

Admite NULL: Sí

Descripción:

Valor nuevo.

changed_by

Tipo: varchar(50)

Descripción:

Usuario o proceso responsable del cambio.

changed_at

Tipo: datetime

Descripción:

Fecha y hora del cambio.